

Materialized Views in PostgreSQL

Table of Contents

- 1. [Learning Objectives](#)
- 2. [What Are Materialized Views?](#)
- 3. [Creating Materialized Views](#)
- 4. [REFRESH MATERIALIZED VIEW](#)
- 5. [REFRESH CONCURRENTLY](#)
- 6. [When to Use Materialized Views](#)
- 7. [Indexing Materialized Views](#)
- 8. [Incremental Refresh Patterns](#)
- 9. [Dependency Management](#)
- 10. [Real-World Use Cases](#)
- 11. [Production Monitoring Queries](#)
- 12. [Common Mistakes](#)
- 13. [Best Practices](#)
- 14. [Performance Considerations](#)
- 15. [Interview Questions & Answers](#)
- 16. [Exercises and Solutions](#)
- 17. [Cross-References](#)

Learning Objectives

By the end of this module you will be able to:

- Explain the architectural difference between views, materialized views, and tables
- Create and refresh materialized views with and without the CONCURRENTLY option
- Design incremental refresh patterns for near-real-time materialized data
- Index materialized views for fast lookups
- Implement automated refresh schedules using pg_cron or triggers
- Diagnose stale data issues and design refresh strategies

What Are Materialized Views?

Regular View vs Materialized View:

Regular View

```
SELECT name
FROM my_view;
```



Re-executes the full query every time (no data stored)
--

Materialized View

```
SELECT name
FROM my_mat_view;
```



Reads from cached snapshot on disk (real heap table) Stale until REFRESH

Slow (on complex queries)

Fast, but not always fresh

A materialized view stores the result of a query as a physical table on disk. Unlike a regular view (which re-executes the query every time), a materialized view:

- Stores actual rows in a heap relation
- Supports indexes
- Is NOT automatically updated when base tables change
- Requires explicit `REFRESH MATERIALIZED VIEW` to update

Creating Materialized Views

Basic syntax

```
CREATE MATERIALIZED VIEW [ IF NOT EXISTS ] view_name
    [ (column_names) ]
    [ USING tablespace_name ]
    [ WITH (storage_parameter = value) ]
AS query
[ WITH [ NO ] DATA ];
```

Example: E-commerce dashboard

```
-- Base tables
CREATE TABLE orders (
    id          BIGSERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    status      TEXT NOT NULL,
    total       NUMERIC(12,2) NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_items (
    id          BIGSERIAL PRIMARY KEY,
    order_id    BIGINT REFERENCES orders(id),
    product_id  INT NOT NULL,
    quantity    INT NOT NULL,
    unit_price  NUMERIC(10,2) NOT NULL
);

CREATE TABLE products (
    id          INT PRIMARY KEY,
    name        TEXT NOT NULL,
    category    TEXT NOT NULL,
    cost        NUMERIC(10,2)
);

-- Populate with test data
INSERT INTO products (id, name, category, cost) VALUES
```

```

(1, 'Widget A', 'electronics', 10.00),
(2, 'Widget B', 'clothing', 5.00),
(3, 'Widget C', 'electronics', 25.00);

INSERT INTO orders (customer_id, status, total, created_at)
SELECT
    (random()*1000)::int + 1,
    CASE (random()*2)::int WHEN 0 THEN 'completed' WHEN 1 THEN 'pending' ELSE
'cancelled' END,
    ROUND((random()*500)::numeric, 2),
    now() - (random()*365)::int * interval '1 day'
FROM generate_series(1, 50000);

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    (random()*50000)::int + 1,
    (random()*3)::int + 1,
    (random()*5)::int + 1,
    ROUND((random()*100)::numeric, 2)
FROM generate_series(1, 200000);

-- Materialized view: daily sales summary
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
    DATE_TRUNC('day', o.created_at)::date AS sale_date,
    COUNT(DISTINCT o.id) AS order_count,
    COUNT(DISTINCT o.customer_id) AS unique_customers,
    SUM(oi.quantity) AS total_units,
    SUM(oi.quantity * oi.unit_price) AS gross_revenue,
    SUM(oi.quantity * (oi.unit_price - p.cost)) AS gross_profit
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN products p ON p.id = oi.product_id
WHERE o.status = 'completed'
GROUP BY DATE_TRUNC('day', o.created_at)::date
ORDER BY sale_date
WITH DATA;

-- Query the materialized view (fast!)
SELECT sale_date, gross_revenue
FROM daily_sales_summary
WHERE sale_date >= CURRENT_DATE - 30
ORDER BY sale_date;

-- Create WITHOUT DATA (empty; populate later)
CREATE MATERIALIZED VIEW product_category_stats AS
SELECT
    p.category,
    COUNT(DISTINCT oi.order_id) AS orders_count,
    SUM(oi.quantity) AS total_sold,
    AVG(oi.unit_price) AS avg_price
FROM order_items oi

```

```
JOIN products p ON p.id = oi.product_id
GROUP BY p.category
WITH NO DATA;

-- Populate later
REFRESH MATERIALIZED VIEW product_category_stats;
```

REFRESH MATERIALIZED VIEW

```
-- Full refresh: drops all data, repopulates (acquires EXCLUSIVE lock)
REFRESH MATERIALIZED VIEW daily_sales_summary;

-- With time measurement
\timing on
REFRESH MATERIALIZED VIEW daily_sales_summary;
-- Time: 342.851 ms

-- Check last refresh time
SELECT
    schemaname,
    matviewname,
    pg_size_pretty(pg_total_relation_size(matviewname::regclass)) AS size,
    hasindexes,
    ispopulated
FROM pg_matviews
WHERE matviewname = 'daily_sales_summary';

-- Refresh in a transaction (useful for coordinated refreshes)
BEGIN;
    REFRESH MATERIALIZED VIEW daily_sales_summary;
    REFRESH MATERIALIZED VIEW product_category_stats;
COMMIT;

-- Both views refreshed atomically (readers see old data until COMMIT)
```

Lock implications

REFRESH MATERIALIZED VIEW (without CONCURRENTLY):

Takes: EXCLUSIVE lock on materialized view
Blocks: All reads AND writes
Duration: Entire refresh time
Impact: Unacceptable for production (readers blocked)

REFRESH CONCURRENTLY

```
-- Prerequisite: UNIQUE index must exist
CREATE UNIQUE INDEX ON daily_sales_summary (sale_date);

-- Now concurrent refresh works
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary;
```

How CONCURRENTLY works internally

REFRESH MATERIALIZED VIEW CONCURRENTLY:

Step 1: Compute new data into a TEMP table
Step 2: Diff new vs old data (INSERT/UPDATE/DELETE rows)
Step 3: Apply changes with row-level locks only
Step 4: Clean up temp table

Takes: SHARE UPDATE EXCLUSIVE lock (not EXCLUSIVE)
Blocks: Other concurrent refreshes only
Does NOT block: Regular reads
Cost: More I/O (reads old data for diff)

```
-- CONCURRENTLY requirements:
-- 1. Must have at least one UNIQUE index
-- 2. Takes longer than non-concurrent (does a diff)
-- 3. Requires more temporary disk space
-- 4. Cannot run inside explicit transaction block

-- Performance comparison
EXPLAIN (ANALYZE, BUFFERS)
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary;
-- Shows: actual rows diffed, time spent

-- Monitor a long-running concurrent refresh
SELECT
    pid,
    state,
    wait_event_type,
    wait_event,
    query,
    now() - query_start AS duration
FROM pg_stat_activity
WHERE query LIKE '%REFRESH%';
```

When to Use Materialized Views

Use materialized views when:

Decision Matrix:

		Query complexity	
		Simple	Complex
Refresh frequency	High	Regular View	Matview if < 1s
	Low	Regular View	Matview Ideal!

Good candidates:

- Dashboard/reporting queries (aggregations, multi-table joins)
- Pre-computed ML features (daily user activity scores)
- API response caching (product catalog with prices/inventory)
- Full-text search indexes across multiple tables
- Geospatial aggregations (PostGIS)

Poor candidates:

- Queries that need real-time data
- Simple single-table queries (just add an index)
- Tables updated every second (refresh overhead exceeds gain)
- Data that changes faster than refresh time

Indexing Materialized Views

Materialized views support full index capabilities since they are physical tables.

```
-- B-tree index on most common filter column
CREATE INDEX ON daily_sales_summary (sale_date);

-- Unique index (required for CONCURRENTLY)
CREATE UNIQUE INDEX ON daily_sales_summary (sale_date);

-- Composite index
CREATE INDEX ON daily_sales_summary (sale_date, gross_revenue DESC);

-- Partial index (only index recent data)
CREATE INDEX ON daily_sales_summary (sale_date)
WHERE sale_date >= CURRENT_DATE - 90;

-- Check index usage on materialized view
SELECT
    indexname,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size
```

```
FROM pg_stat_user_indexes
WHERE relname = 'daily_sales_summary';
```

Incremental Refresh Patterns

PostgreSQL does not natively support incremental (partial) refresh — `REFRESH` always rebuilds from scratch. Here are patterns to work around this:

Pattern 1: Sliding-window materialized view

```
-- Instead of refreshing all history, only maintain recent data
CREATE MATERIALIZED VIEW recent_orders_stats AS
SELECT
    customer_id,
    COUNT(*) AS order_count,
    SUM(total) AS lifetime_value,
    MAX(created_at) AS last_order_at
FROM orders
WHERE created_at >= now() - interval '90 days'
GROUP BY customer_id
WITH DATA;

CREATE UNIQUE INDEX ON recent_orders_stats (customer_id);

-- Refresh is fast because it only processes 90 days
REFRESH MATERIALIZED VIEW CONCURRENTLY recent_orders_stats;
```

Pattern 2: Partitioned matview + swap

```
-- Maintain current-month and historical separately
CREATE MATERIALIZED VIEW sales_summary_current_month AS
SELECT
    DATE_TRUNC('day', created_at)::date AS sale_date,
    SUM(total) AS revenue
FROM orders
WHERE created_at >= DATE_TRUNC('month', now())
    AND status = 'completed'
GROUP BY 1
WITH DATA;

CREATE MATERIALIZED VIEW sales_summary_historical AS
SELECT
    DATE_TRUNC('day', created_at)::date AS sale_date,
    SUM(total) AS revenue
FROM orders
WHERE created_at < DATE_TRUNC('month', now())
    AND status = 'completed'
GROUP BY 1
WITH DATA;
```

```

-- Only refresh current month (fast)
REFRESH MATERIALIZED VIEW CONCURRENTLY sales_summary_current_month;

-- Refresh historical monthly (slow but infrequent)
REFRESH MATERIALIZED VIEW sales_summary_historical;

-- Unified view over both (no materialization needed here)
CREATE OR REPLACE VIEW sales_summary AS
    SELECT * FROM sales_summary_historical
    UNION ALL
    SELECT * FROM sales_summary_current_month;

```

Pattern 3: Delta tracking with changelog table

```

-- Track changes since last refresh
CREATE TABLE orders_changelog (
    order_id    BIGINT,
    changed_at  TIMESTAMPTZ DEFAULT now(),
    operation   TEXT -- INSERT, UPDATE, DELETE
);

CREATE OR REPLACE FUNCTION orders_change_trigger()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO orders_changelog (order_id, operation)
    VALUES (COALESCE(NEW.id, OLD.id), TG_OP);
    RETURN COALESCE(NEW, OLD);
END;
$$;

CREATE TRIGGER trg_orders_change
AFTER INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION orders_change_trigger();

-- Smart refresh: only if changes exist
CREATE OR REPLACE PROCEDURE smart_refresh_sales() LANGUAGE plpgsql AS $$
DECLARE
    v_changes INT;
BEGIN
    SELECT COUNT(*) INTO v_changes FROM orders_changelog;
    IF v_changes > 0 THEN
        REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary;
        DELETE FROM orders_changelog;
        RAISE NOTICE 'Refreshed based on % changes', v_changes;
    ELSE
        RAISE NOTICE 'No changes, skipping refresh';
    END IF;
END;
$$;

```


Pattern 4: pg_cron scheduled refresh

```
-- Auto-refresh every 5 minutes
SELECT cron.schedule(
    'refresh-sales-summary',
    '* /5 * * * *', -- every 5 minutes
    $$REFRESH MATERIALIZED VIEW CONCURRENTLY daily_sales_summary$$
);

-- Refresh at 3 AM daily for heavy aggregation
SELECT cron.schedule(
    'refresh-historical-sales',
    '0 3 * * *',
    $$REFRESH MATERIALIZED VIEW sales_summary_historical$$
);

-- Check scheduled jobs
SELECT * FROM cron.job;
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 10;
```

Dependency Management

```
-- Materialized views can depend on other views/matviews
CREATE MATERIALIZED VIEW hourly_revenue AS
SELECT
    DATE_TRUNC('hour', created_at) AS hour,
    SUM(total) AS revenue
FROM orders WHERE status = 'completed'
GROUP BY 1;

CREATE UNIQUE INDEX ON hourly_revenue (hour);

-- Matview depending on another matview
CREATE MATERIALIZED VIEW daily_revenue AS
SELECT
    DATE_TRUNC('day', hour)::date AS day,
    SUM(revenue) AS revenue
FROM hourly_revenue
GROUP BY 1;

-- You must refresh in dependency order!
REFRESH MATERIALIZED VIEW hourly_revenue;
REFRESH MATERIALIZED VIEW daily_revenue; -- uses fresh hourly data

-- Check dependencies
SELECT DISTINCT
    dependent_view.relname AS dependent,
    source_view.relname   AS depends_on
FROM pg_depend
```

```

JOIN pg_class dependent_view ON dependent_view.oid = pg_depend.objid
JOIN pg_class source_view    ON source_view.oid    = pg_depend.refobjid
WHERE dependent_view.relkind = 'm'
      AND source_view.relkind IN ('m', 'v', 'r')
ORDER BY 1;

-- Drop a materialized view and its dependents
DROP MATERIALIZED VIEW IF EXISTS hourly_revenue CASCADE;
-- This also drops daily_revenue (which depends on hourly_revenue)

```

Real-World Use Cases

1. Product Catalog with Pre-computed Rankings

```

CREATE MATERIALIZED VIEW product_rankings AS
SELECT
  p.id,
  p.name,
  p.category,
  COUNT(oi.id)           AS total_orders,
  SUM(oi.quantity)       AS total_sold,
  AVG(oi.unit_price)     AS avg_price,
  SUM(oi.quantity * oi.unit_price) AS revenue,
  RANK() OVER (
    PARTITION BY p.category
    ORDER BY SUM(oi.quantity) DESC
  ) AS category_rank,
  to_tsvector('english', p.name) AS search_vector
FROM products p
LEFT JOIN order_items oi ON oi.product_id = p.id
GROUP BY p.id, p.name, p.category
WITH DATA;

CREATE UNIQUE INDEX ON product_rankings (id);
CREATE INDEX ON product_rankings (category, category_rank);
CREATE INDEX ON product_rankings USING GIN (search_vector);

-- Fast product search with pre-computed ranking
SELECT id, name, category_rank, total_sold
FROM product_rankings
WHERE search_vector @@ plainto_tsquery('english', 'widget')
      AND category = 'electronics'
ORDER BY category_rank
LIMIT 10;

```

2. User Activity Summary for Personalization

```

CREATE MATERIALIZED VIEW user_activity_summary AS
WITH order_stats AS (

```

```

SELECT
    customer_id,
    COUNT(*) AS total_orders,
    SUM(total) AS lifetime_value,
    MAX(created_at) AS last_order_at,
    AVG(total) AS avg_order_value,
    MIN(created_at) AS first_order_at
FROM orders WHERE status = 'completed'
GROUP BY customer_id
),
category_prefs AS (
    SELECT
        o.customer_id,
        p.category,
        SUM(oi.quantity) AS qty_by_category,
        RANK() OVER (PARTITION BY o.customer_id ORDER BY SUM(oi.quantity) DESC)
        AS pref_rank
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    JOIN products p ON p.id = oi.product_id
    WHERE o.status = 'completed'
    GROUP BY o.customer_id, p.category
)
SELECT
    os.customer_id,
    os.total_orders,
    os.lifetime_value,
    os.avg_order_value,
    os.last_order_at,
    os.first_order_at,
    cp.category AS top_category,
    CASE
        WHEN os.lifetime_value > 1000 THEN 'gold'
        WHEN os.lifetime_value > 500 THEN 'silver'
        ELSE 'bronze'
    END AS tier
FROM order_stats os
LEFT JOIN category_prefs cp ON cp.customer_id = os.customer_id AND cp.pref_rank = 1
WITH DATA;

CREATE UNIQUE INDEX ON user_activity_summary (customer_id);
CREATE INDEX ON user_activity_summary (tier, lifetime_value DESC);

```

Production Monitoring Queries

```

-- List all materialized views with size and refresh status
SELECT
    schemaname,
    matviewname,
    pg_size_pretty(pg_total_relation_size(matviewname::regclass)) AS total_size,

```

```

        pg_size_pretty(pg_relation_size(matviewname::regclass))      AS data_size,
        ispopulated,
        hasindexes
FROM pg_matviews
ORDER BY pg_total_relation_size(matviewname::regclass) DESC;

-- Check if any matview is being refreshed right now
SELECT pid, state, query, now() - query_start AS duration
FROM pg_stat_activity
WHERE query ILIKE '%refresh materialized%';

-- Find matviews that haven't been refreshed recently
-- (pg_matviews doesn't store last refresh time – use a custom tracking table)
CREATE TABLE matview_refresh_log (
    matview_name TEXT,
    refreshed_at TIMESTAMPTZ DEFAULT now(),
    duration_ms  NUMERIC,
    row_count    BIGINT
);

-- Wrapper function that logs refresh time
CREATE OR REPLACE FUNCTION refresh_and_log(p_matview TEXT)
RETURNS void LANGUAGE plpgsql AS $$
DECLARE
    v_start TIMESTAMPTZ := clock_timestamp();
    v_count  BIGINT;
BEGIN
    EXECUTE format('REFRESH MATERIALIZED VIEW CONCURRENTLY %I', p_matview);
    EXECUTE format('SELECT COUNT(*) FROM %I', p_matview) INTO v_count;
    INSERT INTO matview_refresh_log (matview_name, duration_ms, row_count)
    VALUES (p_matview,
            EXTRACT(EPOCH FROM (clock_timestamp() - v_start)) * 1000,
            v_count);
END;
$$;

SELECT refresh_and_log('daily_sales_summary');

-- Recent refresh history
SELECT matview_name, refreshed_at, duration_ms, row_count
FROM matview_refresh_log
ORDER BY refreshed_at DESC
LIMIT 20;

-- Check if indexes on matviews are being used
SELECT
    indexname,
    tablename,
    idx_scan,
    idx_tup_read,
    pg_size_pretty(pg_relation_size(indexrelid)) AS idx_size
FROM pg_stat_user_indexes

```

```
WHERE tablename IN (SELECT matviewname FROM pg_matviews)
ORDER BY idx_scan DESC;
```

Common Mistakes

1. **Using `REFRESH` without `CONCURRENTLY` in production** — This takes an `EXCLUSIVE` lock, blocking all queries against the materialized view for the entire refresh duration. Always use `CONCURRENTLY` in production.
2. **Forgetting to create a `UNIQUE` index before using `CONCURRENTLY`** — `REFRESH CONCURRENTLY` requires at least one unique index. Without it, you get: `ERROR: cannot refresh materialized view concurrently without a unique index`.
3. **Not refreshing dependency chain in order** — If matview B depends on matview A, refresh A first, then B. Out-of-order refreshes use stale data from A.
4. **Refreshing too frequently** — If refresh takes 30 seconds, scheduling every minute means a permanent refresh loop (previous refresh hasn't finished when next starts). Monitor duration and set `interval > 2 × refresh time`.
5. **Not handling the `WITH NO DATA` case in application code** — `ispopulated = false` means querying the view returns no rows. Always check `ispopulated` after creation.
6. **Materializing views that reference `now()` or `CURRENT_DATE`** — The snapshot is fixed at refresh time. Queries like `WHERE created_at > now() - interval '7 days'` in the materialized view definition won't use current time on each query; they use the time the refresh ran.

Best Practices

1. **Always use `CONCURRENTLY` in production** after creating a unique index.
2. **Measure refresh time** with `matview_refresh_log` and set schedule accordingly.
3. **Use partial matviews for recent data** — Refresh fast (recent window) + slow (full history) on different schedules.
4. **Index every column you filter on** — Matviews are just tables; B-tree and GIN indexes work normally.
5. **Document staleness tolerance** in application code — Make it explicit that this data is up to N minutes stale.
6. **Use `EXPLAIN` on the refresh** — `EXPLAIN (ANALYZE) REFRESH ...` shows you where refresh time is spent.

Performance Considerations

```
-- Compare: view query vs matview query
\timing on
-- Regular view (re-runs the complex join every time)
SELECT * FROM v_daily_sales_summary WHERE sale_date = CURRENT_DATE;
-- Time: 2847 ms

-- Materialized view with index
SELECT * FROM daily_sales_summary WHERE sale_date = CURRENT_DATE;
```

```
-- Time: 0.3 ms

-- Size vs freshness tradeoff
-- A matview that aggregates 10M rows might be 1MB after aggregation
-- Querying 1MB is fast; re-reading 10M rows every query is not

-- Vacuum materialized views (they accumulate dead tuples from concurrent refresh)
VACUUM ANALYZE daily_sales_summary;

-- Check matview bloat
SELECT
    schemaname || '.' || relname AS matview,
    n_dead_tup,
    n_live_tup,
    ROUND(n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup, 0) * 100, 1) AS
dead_pct
FROM pg_stat_user_tables
WHERE relname IN (SELECT matviewname FROM pg_matviews)
ORDER BY dead_pct DESC;
```

Interview Questions & Answers

Q1: What is the fundamental difference between a view and a materialized view?

A: A regular view is a stored query — it stores no data and re-executes the underlying query every time it's accessed. A materialized view stores the actual result set on disk as a physical table. Views always return fresh data but are slow for complex queries; materialized views are fast but can return stale data until explicitly refreshed.

Q2: Why does `REFRESH MATERIALIZED VIEW CONCURRENTLY` require a unique index?

A: `CONCURRENTLY` works by computing new data into a temp table, then diffing it against the existing materialized view data to identify rows to `INSERT`, `UPDATE`, or `DELETE`. This diff requires the ability to uniquely identify each row — which requires a unique index. Without it, PostgreSQL can't determine which rows are "the same" in old vs new data.

Q3: What locks does `REFRESH MATERIALIZED VIEW` take, and why does it matter?

A: Regular `REFRESH` takes an `EXCLUSIVE` lock, blocking all reads and writes for the duration of the refresh. `REFRESH CONCURRENTLY` takes only a `SHARE UPDATE EXCLUSIVE` lock, which blocks other concurrent refreshes but allows reads and writes to proceed. In production, always use `CONCURRENTLY` to avoid blocking application queries.

Q4: How would you implement near-real-time data in a materialized view?

A: Several approaches: (1) Refresh very frequently (every 30 seconds) using `pg_cron` if refresh time is sub-second. (2) Use a trigger-driven changelog and refresh only when enough changes accumulate. (3) Combine a historical materialized view (refreshed slowly) with a live view of recent data, unioned together. (4) Consider whether a regular view with good indexes would suffice — sometimes the base query is fast enough.

Q5: Can you write to a materialized view?

A: No. Materialized views are read-only. You cannot INSERT, UPDATE, or DELETE from them directly. All data comes from the REFRESH process. To modify data, update the base tables and refresh the materialized view.

Q6: How do you handle materialized views in a multi-schema deployment?

A: Use schema-qualified names and ensure the refresh user has SELECT on base tables and TRIGGER permissions if using change tracking. In multi-tenant systems, consider per-tenant materialized views in separate schemas or a single shared materialized view with a tenant_id column. Remember that refresh runs with the privileges of the user who calls REFRESH.

Q7: What happens if the base table structure changes (ALTER TABLE)?

A: The materialized view definition is fixed at creation time. If you add a column to the base table, the matview won't include it automatically. You must DROP and recreate the materialized view. This is a major operational consideration — schema changes require a matview recreation plan.

Q8: How do you estimate the refresh performance impact on the database?

A: (1) Check pg_stat_statements for the underlying query cost. (2) Run EXPLAIN (ANALYZE, BUFFERS) on the REFRESH. (3) Monitor CPU, I/O, and lock waits during refresh window. (4) A concurrent refresh touches old data + new data — double the I/O compared to a clean refresh. (5) Schedule refreshes during low-traffic windows when possible.

Q9: Describe a scenario where a materialized view would be the wrong choice.

A: If the data changes every few seconds and the application requires data to be < 1 second stale, materialized views are wrong — the refresh overhead and latency make it impractical. Use a table with good indexes, a caching layer (Redis), or LISTEN/NOTIFY for event-driven updates instead. Also wrong for simple queries on small tables where a regular view + index is fast enough.

Q10: How do you migrate from a regular view to a materialized view without downtime?

A: (1) Create the new materialized view with a different name. (2) Add indexes. (3) Set up a refresh schedule. (4) Verify query plans and performance. (5) Rename: DROP the old view and rename the materialized view to the old name (or update application queries). (6) All done — the materialized view now serves the same name. Application code requires no changes if the column names match.

Exercises and Solutions

Exercise 1

Create a materialized view that summarizes monthly revenue per product category. Add a unique index. Refresh it concurrently and verify with a SELECT.

Solution:

```
CREATE MATERIALIZED VIEW monthly_category_revenue AS
SELECT
    DATE_TRUNC('month', o.created_at)::date AS month,
    p.category,
    SUM(oi.quantity * oi.unit_price)      AS revenue,
    COUNT(DISTINCT oi.id)                 AS order_count
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
```

```

JOIN products p ON p.id = oi.product_id
WHERE o.status = 'completed'
GROUP BY 1, 2
WITH DATA;

CREATE UNIQUE INDEX ON monthly_category_revenue (month, category);

REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_category_revenue;

SELECT * FROM monthly_category_revenue ORDER BY month DESC, revenue DESC;

```

Exercise 2

Write a `smart_refresh` function that: (1) checks if the base table has been modified since last refresh (use a timestamp table), (2) only refreshes if it has, (3) logs the result.

Solution:

```

CREATE TABLE refresh_tracker (
    matview_name TEXT PRIMARY KEY,
    last_refresh TIMESTAMPTZ DEFAULT '-infinity'
);

CREATE TABLE orders_last_modified AS
    SELECT MAX(created_at) AS ts FROM orders;

CREATE OR REPLACE FUNCTION smart_refresh_monthly_revenue()
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
    v_last_refresh TIMESTAMPTZ;
    v_last_modified TIMESTAMPTZ;
    v_start        TIMESTAMPTZ := clock_timestamp();
BEGIN
    SELECT last_refresh INTO v_last_refresh
    FROM refresh_tracker WHERE matview_name = 'monthly_category_revenue';

    IF v_last_refresh IS NULL THEN
        INSERT INTO refresh_tracker VALUES ('monthly_category_revenue', '-infinity');
        v_last_refresh := '-infinity'::timestamptz;
    END IF;

    SELECT MAX(created_at) INTO v_last_modified FROM orders;

    IF v_last_modified > v_last_refresh THEN
        REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_category_revenue;
        UPDATE refresh_tracker
        SET last_refresh = clock_timestamp()
        WHERE matview_name = 'monthly_category_revenue';
        RETURN format('Refreshed in %s ms',
            ROUND(EXTRACT(EPOCH FROM (clock_timestamp()-v_start))*1000));
    ELSE

```



```
        RETURN 'Skipped – no changes since last refresh';
    END IF;
END;
$$;

SELECT smart_refresh_monthly_revenue();
```

Cross-References

- **03_partitioning.md** — Matviews over partitioned tables
- **02_full_text_search.md** — Materializing `tsvector` columns across tables
- **07_indexes** — Indexing strategies applicable to matviews
- **09_stored_procedures_functions.md** — Trigger patterns for change detection
- **06_maintenance_procedures.md** (12_Production) — `VACUUM` for matviews
- PostgreSQL Docs — <https://www.postgresql.org/docs/current/rules-materializedviews.html>